The Mirror

VOL. 1, NO. 037

2026-03-30

COVER STORY

An Exercise Program for the Fat Web

When I wrote about App-pocalypse Now in 2014, I implied the future still belonged to the web.

Jeff Atwood · Coding Horror (Jeff Atwood)

And it does. But it's also true that the web has changed a lot in the last 10 years, much less the last 20 or 30.

Websites have gotten a lot... fatter .

While I think it's irrational to pine for the bad old days of HTML 1.0 websites , there are some legitimate concerns here. The best summary is Maciej Cegłowski's, The Website Obesity Crisis :

To channel a famous motivational speaker, I could go out there tonight, with the materials you've got, and rewrite the sites I showed you at the start of this talk to make them load in under a second. In two hours.

Can you? Can you?

Of course you can! It's not hard! We knew how to make small websites in 2002. It's not like the secret has been lost to history, like Greek fire or Damascus steel.

But we face pressure to make these sites bloated.

I bet if you went to a client and presented a 200 kilobyte site template, you'd be fired. Even if it looked great and somehow included all the tracking and ads and social media crap they insisted on putting in. It's just so far out of the realm of the imaginable at this point.

The whole article is essential; you should stop what you're doing and read it now if you haven't already. But if you don't have time, here's the key point:

This is a screenshot from an NPR article discussing the rising use of ad blockers. The page is 12 megabytes in size in a stock web browser. The same article with basic ad blocking turned on is 1 megabyte.

That's right, through the simple act of running an ad blocker, you've reduced that website's payload by twelve times. Twelve! That's like the most effective exercise program ever!

Even the traditional advice to keep websites lean and mean for mobile no longer applies because new mobile devices, at least on the Apple side, are faster than most existing desktops and laptops.

Despite claims to the contrary , the bad guy isn't web bloat, per se. The bad guy is advertising . Unlimited, unfettered ad "tech" has creeped into everything and subsumed the web.

Personally I don't even want to run ad blockers, and I didn't for a long time – but it's increasingly difficult to avoid running an ad blocker unless you want a clunky, substandard web experience. There's a reason the most popular browser plugins are inevitably ad blockers, isn't there? Just ask Google:

So it's all the more surprising to learn that Google is suddenly clamping down hard on adblockers in Chrome. Here's what the author of uBlock Origin, an ad blocking plugin for Chrome, has to say about today's announcement:

In order for Google Chrome to reach its current user base, it had to support content blockers – these are the top most popular extensions for any browser. Google strategy has been to find the optimal point between the two goals of growing the user base of Google Chrome and preventing content blockers from harming its business.

The blocking ability of the webRequest API caused Google to yield control of content blocking to content blockers. Now that Google Chrome is the dominant browser, it is in a better position to shift the optimal point between the two goals which benefits Google's primary business.

The deprecation of the blocking ability of the webRequest API is to gain back this control, and to further instrument and report how web pages are filtered, since the exact filters which are applied to web pages are useful information which will be collectable by Google Chrome.

The ad blockers themselves are arguably just as complicit. Eye/o GmbH owns AdBlock and uBlock , employs 150 people, and in 2016 they had 50 million euros in revenue, of which about 50% was profit. Google's paid "Acceptable Ads" program is a way to funnel money into adblockers to, uh, encourage them to display certain ads. With money. Lots... and lots... of money.

We simultaneously have a very real web obesity crisis, and a looming crackdown on ad blockers, seemingly the only viable weight loss program for websites. What's a poor web citizen to do? Well, there is one thing you can do to escape the need for browser-based adblockers, at least on your home network. Install and configure Pi-Hole .

I've talked about the amazing Raspberry Pi before in the context of classic game emulation , but this is another brilliant use for a Pi.

Here's why it's so cool. If you disable the DHCP server on your router, and let the Pi-Hole become your primary DHCP server, you get automatic DNS based blocking of ads for every single device on your network . It's kind of scary how powerful DNS can be, isn't it?

My Pi-Hole took me about 1 hour to set up, start to finish. All you need is

a Raspberry Pi 3b+ kit $59

a quality 32GB SD card $9

an ethernet cable

I do recommend the 3b+ because it has native gigabit ethernet and a bit more muscle. But literally any Raspberry Pi you can find laying around will work, though I'd strongly advise you to pick one with a wired ethernet port since it'll be your DNS server.

I'm not going to write a whole Pi-Hole installation guide , because there are lots of great ones out there already. It's not difficult, and there's a slick web GUI waiting for you once you complete initial setup. For your initial testing, pick any IP address you like on your network that won't conflict with anything active. Once you're happy with the basic setup and web interface:

Turn OFF your router's DHCP server – existing leases will continue to work, so nothing will be immediately broken.

Turn ON the pi-hole DHCP server, in the web GUI.

Once you do this, all your network devices will start to grab their DHCP leases from your Pi-Hole, which will also tell them to route all their DNS requests through the Pi-Hole, and that's when the magic happens!

All those DNS requests from all the devices on your network will be checked against the ad blacklists; anything matching is quickly and silently discarded before it ever reaches your browser.

(The Pi-Hole also acts as a caching DNS server , so repeated DNS requests will be serviced rapidly from your local network, too.)

If you're worried about stability or reliability, you can easily add a cheap battery backed USB plug, or even a second backup Pi-Hole as your secondary DNS provider if you prefer belt and suspenders protection. Switching back to plain boring old vanilla DNS is as easy as unplugging the Pi and flicking the DHCP server setting in your router back on.

At this point if you're interested (and you should be!), just give it a try. If you're looking for more information, the project has an excellent forum full of FAQs and roadmaps.

You can even vote for your favorite upcoming features !

I avoided the Pi-Hole project for a while because I didn't need it, and I'd honestly rather jump in later when things are more mature.

With the latest Chrome crackdown on ad blockers, now is the time, and I'm impressed how simple and easy Pi-Hole is to run. Just find a quiet place to plug it in, spend an hour configuring it, and promptly proceed to forget about it forever as you enjoy a lifetime subscription to a glorious web ad instant weight loss program across every single device on your network with (almost) zero effort!

Finally, an exercise program I can believe in.

{

FRONT PAGE
Is the craft dead?

Scott Hanselman

The Japanese are really good at woodworking. And I love watching the Yankee workshop, my dad makes Native American bows and arrows completely from scratch in his workshop with trees that he finds.

This is all different from the stuff you get at IKEA, but I've been coding now for money for 35 years and systems are still complicated, computers still do dumb stuff, humans still do dumb stuff, this is just like the move from assembler to C, like the introduction of syntax highlighting, the introduction of intellisense, and the copy paste directly into production shift when stack overflow happened.

There is value in good taste, there is value in craftsmanship, and there is value in human judgment. The furniture might be differently designed, but we're still interior designers and putting together a cohesive system is non-trivial.

Don't let them gaslight you with one shot Minecraft clones and one shot C compilers. Software is still hard, it's just that you're no longer I/O bound with the speed of your fingertips.

I think that there will be lots of work for us cleaning up after the slop, but if you know what you're doing AI augmented development is going to get you some amazing results and I am enjoying learning a ton during this momentous era shift - but the craft still exists.

style="clear: both; padding-top: 0.2em;">

}

{

FEATURES

The Gumbel-Max Trick Made Intuitive

Criteo Tech · Criteo Engineering

Author: Alexandre Gilotte

When we work with machine learning models, we constantly turn probabilities into discrete choices.

Which ad do we show? Which action do we sample in a reinforcement learning policy? Which word comes next in a language model?

All these problems share the same core operation:

We have a list of options, each with an associated probability, and we want to randomly pick one option according to those probabilities.

We will use this simple distribution to illustrate why Gumbel trick works

There is a very common way to do this (using cumulative sums and a uniform random number), but one could instead imagine an algorithm that looks like this:

Independently noise each option's score, then simply take the argmax of the noised scores.

Here it is not obvious that there is even a "right" way to noise the scores so that we get exactly the correct distribution. It turns out there is one, known as the Gumbel-Max trick :

Take the log of the scores (or probabilities) for each category.

Add independent Gumbel noise to each log-probability.

Pick the index with the largest noisy log-probability.

The magic is that the index you get is exactly a sample from the categorical distribution defined by your original probabilities.

Here, if you have no idea what a "Gumbel" variable is, don't worry. It is just a specific distribution on a continuous random variable; we will get back to it later.

And there is more: we can also draw several samples without replacement from the same categorical distribution with the same trick. For that, simply return the items ordered by their decreasing noisy score. That's all.

The figure below illustrates the sampling on an example:

Example of sampling with Gumbel max trick. (Bold values are random numbers.)

The usual proof that Gumbel-Max works consists of computing some ugly integrals involving the Gumbel pdf, to conclude after a page of calculus that "it outputs each item with the right probability". But this proof gives no intuition about why it works.

In this post, I will:

Reformulate the Gumbel-Max trick with the more commonly used exponential distribution , to get an "exponential-min trick".

Link the exponential to arrival times , using a simple Autobus / Bike / Car example, to get an intuitive explanation of why it works.

Explain why sorting the noisy scores gives sampling without replacement .

From the Gumbel-Max trick to the Exp-Min trick

I have personally never encountered a "Gumbel" random variable except in this "Gumbel-Max trick". Fortunately, we can define it using the more familiar Exponential distribution.

Gumbel: What is it? Take an exponential random variable with rate 1, written X Exp(1). Define:

G = −log X

Then G has the standard Gumbel distribution.

As a reminder, one way to generate X Exp(1) is:

draw U Uniform(0, 1)

set X := −log U

This means G can be sampled as G := −log(-log(U))

Gumbel variable

Let's see how we can use this definition to reformulate the Gumbel-Max trick without ever mentioning Gumbel again

Gumbel-Max trick algorithm:

Draw an independent Gumbel G_i for each item i.

Return: $\text{Argmax}_i [\log p_i + G_i]$.

From the Gumbel definition: $G_i = -\log X_i$ with X_i Exp(1)

Noting that:

$\log p_i + G_i = -\log(X_i / p_i)$.

−log(·) is strictly decreasing,

therefore the index that maximizes $(\log p_i + G_i)$ is the same index that minimizes (X_i / p_i). So we obtain the Exp-Min trick :

Draw independent X_i Exp(1) for each item i.

Return: $\text{Argmin}_i [X_i / p_i]$.

```
}
{
```

Ending the “silent drop”: how Dynamic Path MTU Discovery makes the Cloudflare One Client more resilient

Koko Uko · Cloudflare Blog

You’ve likely seen this support ticket countless times: a user’s Internet connection that worked just fine a moment ago for Slack and DNS lookups is suddenly hung the moment they attempt a large file upload, join a video call, or initiate an SSH session. The culprit isn’t usually a bandwidth shortage or service outage issue, it is the “PMTUD Black Hole” — a frustration that occurs when packets are too large for a specific network path, but the network fails to communicate that limit back to the sender. This situation often happens when you’re locked into using networks you do not manage or vendors with maximum transmission unit (MTU) restrictions, and you have no means to address the problem.

Today, we are moving past these legacy networking constraints. By implementing Path MTU Discovery (PMTUD), the Cloudflare One Client has shifted from a passive observer to an active participant in path discovery.

Dynamic Path MTU Discovery allows the client to intelligently and dynamically adjust to the optimal packet size for most network paths using MTUs above 1281 bytes . This ensures that a user’s connection remains stable, whether they are on a high-speed corporate backbone or a restrictive cellular network.

The “modern security meets legacy infrastructure” challenge

To understand the solution, we have to look at how modern security protocols interact with the diversity of global Internet infrastructure. The MTU represents the largest data packet size a device can send over a network without fragmentation: typically 1500 bytes for standard Ethernet.

As the Cloudflare One client has evolved to support modern enterprise-grade requirements (such as FIPS 140-2 compliance), the amount of metadata and encryption overhead within each packet has naturally increased. This is a deliberate choice to ensure our users have the highest level of protection available today.

However, much of the world’s Internet infrastructure was built decades ago with a rigid expectation of 1500-byte packets. On specialized networks like LTE/5G, satellite links, or public safety networks like FirstNet, the actual available space for data is often lower than the standard. When a secure, encrypted packet hits an older router with a lower limit (e.g., 1300 bytes), that router should ideally send an Internet Control Message Protocol (ICMP) message stating “Destination Unreachable” back to the sender to request a smaller size.

But that doesn’t always happen. The “Black Hole” occurs when firewalls or middleboxes silently drop those ICMP feedback messages. Without this feedback, the sender keeps trying to send large packets that never arrive, and the application simply waits in a “zombie” state until the connection eventually times out.

Cloudflare’s solution: active probing with PMTUD

Cloudflare’s implementation of RFC 8899 Datagram Packetization Layer Path MTU Discovery (PMTUD) removes the reliance on these fragile, legacy feedback loops. Because our modern client utilizes the MASQUE protocol — built on top of Cloudflare’s open source QUIC library — the client can perform active, end-to-end interrogation of the network path.

Instead of waiting for an error message that might never come, the client proactively sends encrypted packets of varying sizes to the Cloudflare edge. This probe tests MTUs from the upper bound of the supported MTU range to the midpoint, until the client narrows down to the exact MTU to match. This is a sophisticated, non-disruptive handshake happening in the background. If the Cloudflare edge receives a specific-sized probe, it acknowledges it; if a probe is lost, the client instantly knows the precise capacity of that specific network segment.

The client then dynamically resizes its virtual interface MTU on the fly, by periodically validating the capacity of the path that we established at connection onset. This ensures that if, for example, a user moves from a 1500-MTU Wi-Fi network at a station to a 1300-MTU cellular backhaul in the field, the transition is seamless. The application session remains uninterrupted because the client has already negotiated the best possible path for those secure packets.

Real-world impact, from first responders to hybrid workers

This technical shift has profound implications for mission-critical connectivity. Consider the reliability needs of a first responder using a vehicle-mounted router. These systems often navigate complex NAT-traversal and priority-routing layers that aggressively shrink the available MTU. Without PMTUD, critical software like Computer Aided Dispatch (CAD) systems may experience frequent disconnects during tower handoffs or signal fluctuations. By using active discovery, the Cloudflare One Client maintains a sticky connection that shields the application from the underlying network volatility.

This same logic applies to the global hybrid workforce. A road warrior working from a hotel in a different country often encounters legacy middleboxes and complex double-NAT environments. Instead of choppy video calls and stalled file transfers, the client identifies the bottleneck in seconds and optimizes the packet flow — before the user even notices a change.

Get PMTUD for your devices

Anyone using the Cloudflare One Client with the MASQUE protocol can try Path MTU Discovery now for free. Use

```
}
{
```

Launching The Rural Guaranteed Minimum Income Initiative

Jeff Atwood · Coding Horror (Jeff Atwood)

It's been a year since I invited Americans to join us in a pledge to Share the American Dream :

1. Support organizations you feel are effectively helping those most in need across America right now .

2. Within the next five years, also contribute public dedications of time or funds towards longer term efforts to keep the American Dream fair and attainable for all our children.

Stay gold, America.

Personally, I've become a big believer in one particular quote, especially considering the specific context in which it was delivered:

Those 10 words had a profound effect on the world. Indeed, we were given much, so we, as a family, will choose to give much . On a recent podcast , my partner Betsy said it better than I could have:

"Well, we have everything we need!" That's how I've always phrased it to [our children]. That, I think, extends [to our philanthropy]. We have everything we need; how do we make sure everybody has what they need? Because that's the basic thing — Do you have a comfortable place to live? Do you have enough to eat? Do you have healthcare? If you have the basics, you're in a good place in life, and everybody should have that opportunity.

It's a question I've asked myself a lot since 2021 . When, exactly, is enough?

We do have everything we need. Why can't everyone else have the basic things they need, too?

Beyond the $1M to eight nonprofit charities we listed in January 2025, we saw immediate needs becoming so urgent that we quickly added an additional $13M in donations within a few months, for a total of $21M.

value="1"> Team Rubicon — $1M

value="2"> Children's Hunger Fund — $1M

value="3"> PEN America — $1M

value="4"> The Trevor Project — $1M

value="5"> NAACP Legal Defense and Educational Fund — $1M + $100k

value="6"> First Generation Investors — $1M

value="7"> Global Refuge — $1M

value="8"> Planned Parenthood — $1M

value="9"> VoteVets — $2M

value="10"> Mastodon — $1.5M

value="11"> 404 Media — $1.1M

value="12"> Ryan Broderick / Garbage Day — $1M

value="13"> Internet Archive — $1M

value="14"> Common Crawl Foundation — $1M

value="15"> Wikipedia / Wikimedia foundation — $1M

value="16"> Internet Security Research Group — $1M

value="17"> DNA Lounge — $1M

value="18"> Murena — $500k

value="19"> Sharewell — $300k

value="20"> Precious Plastic — $100k

value="21"> Economic Security Project — $100k

value="22"> Rural Democracy Initiative — $100k

value="23"> Civic Nation — $100k

value="24"> Sojourn Project — $750k

value="25"> Alameda Food Bank — $150k

value="26"> Urban Compassion Project — $75k

But you can't take a completely short term view and fight each individual fire reactively, as it comes. You'll never stop firefighting. We also have to do fire abatement and deal with the root causes, improving conditions in this country such that there aren't so many fires . Thus for the second half, much longer term part, in addition to the $21M already donated, we pledged $50M — half of our remaining wealth — to address the underlying, systemic issues.

I proposed some speculative ideas in "Stay Gold," and this one ended up being the closest:

We could found a new organization loosely based on the original RAND Corporation , but modernized like Lever for Change . We can empower the best and brightest to determine a realistic, achievable path toward preserving the American Dream for everyone, working within the current system or outside it.

By March, 2025 we had consensus — The Road Not Taken is Guaranteed Minimum Income .

Guaranteed Minimum Income (GMI) is an improved version of the older concept of Universal Basic Income (UBI) — rather than indiscriminately giving money to "everyone," GMI directs the money towards those who most need it , particularly families experiencing generational poverty.

```
}
{
```

What does Stack Overflow want to be when it grows up?

Jeff Atwood · Coding Horror (Jeff Atwood)

I sometimes get asked by regular people in the actual real world what it is that I do for a living, and here's my 15 second answer:

We built a sort of Wikipedia website for computer programmers to post questions and answers. It's called Stack Overflow .

As of last month, it's been 10 years since Joel Spolsky and I started Stack Overflow . I currently do other stuff now , and I have since 2012, but if I will be known for anything when I'm dead, clearly it is going to be good old Stack Overflow.

Here's where I'd normally segue into a bunch of rah-rah stuff about how great Stack Overflow is, and thus how implicitly great I am by association for being a founder, and all.

I do not care about any of that.

What I do care about, though, is whether Stack Overflow is useful to working programmers . Let's check in with one of my idols , John Carmack. How useful is Stack Overflow, from the perspective of what I consider to be one of the greatest living programmers?

I won't lie, September 17th, 2013 was a pretty good day . I literally got chills when I read that, and not just because I always read the word "billions" in Carl Sagan's voice. It was also pleasantly the opposite of pretty much every other day I'm on Twitter, scrolling through an oppressive, endless litany of shared human suffering and people screaming at each other. Which reminds me, I should check my Twitter and see who else is wrong on the Internet today.

I am honored and humbled by the public utility that Stack Overflow has unlocked for a whole generation of programmers. But I didn't do that .

You did, when you contributed a well researched question to Stack Overflow.

You did, when you contributed a succinct and clear answer to Stack Overflow.

You did, when you edited a question or answer on Stack Overflow to make it better.

All those "fun size" units of Q&A collectively contributed by working programmers from all around the world ended up building a Creative Commons resource that truly rivals Wikipedia within our field. That's... incredible, actually.

But success stories are boring. The world is filled with people that basically got lucky , and subsequently can't stop telling people how it was all of their hard work and moxie that made it happen. I find failure much more instructive, and when building a business and planning for the future, I take on the role of Abyss Domain Expert™ and begin a staring contest. It's just a little something I like to do, you know... for me .

Thus, what I'd like to do right now is peer into that glorious abyss for a bit and introspect about the challenges I see facing Stack Overflow for the next 10 years. Before I begin, I do want to be absolutely crystal clear about a few things:

I have not worked at Stack Overflow in any capacity whatsoever since February 2012 and I've had zero day-to-day operational input since that date, more or less by choice. Do I have opinions about how things should be done? Uh, have you met me? Do I email people every now and then about said opinions? I might, but I honestly do try to keep it to an absolute minimum, and I think my email archive track record here is reasonable.

The people working at Stack are amazing and most of them (including much of the Stack Overflow community, while I'm at it) could articulate the mission better — and perhaps a tad less crankily — than I could by the time I left. Would I trust them with my life? No. But I'd trust them with Joel's life!

The whole point of the Stack Overflow exercise is that it's not beholden to me, or Joel, or any other Great Person . Stack Overflow works because it empowers regular everyday programmers all over the world, just like you, just like me. I guess in my mind it's akin to being a parent. The goal is for your children to eventually grow up to be sane, practicing adults who don't need (or, really, want) you to hang around any more.

Understand that you're reading the weak opinions strongly held the strong opinions weakly held of a co-founder who spent prodigious amounts of time working with the community in the first four years of Stack Overflow's life to shape the rules and norms of the site to fit their needs. These are merely my opinions. I like to think they are informed opinions, but that doesn't necessarily mean I can predict the future, or that I am even qualified to try. But I've never let being "qualified" stop me from doing anything, and I ain't about to start tonight.

id="stack-overflow-is-a-wiki-first">Stack Overflow is a wiki first

Stack Overflow ultimately has much more in common with Wikipedia than a discussion forum. By this I mean questions and answers on Stack Overflow are not primarily judged by their usefulness to a specific individual, but by how many other programmers that question or answer can potentially help over time . I tried as hard as I could to emphasize this relationship from launch day in 2008. Note who has top billing in this Venn diagram.

Stack Overflow later added a super neat feature to highlight this core value in user profiles, where it shows how many other people you have potentially helped with your contributed questions and answers so far.

The most common complaints I see about Stack Overflow

```
}
{
```

Nontraditional Red Teams

Zach Holman

Most developers know about red teams : a specific group of people chosen to be the antagonist to your system, trying to sniff out vulnerabilities in your code or organization. Basically, like Sneakers , or the annoying plotline in The Newsroom season two . (Someone should have really red team'd Sorkin himself on that one.)

There's a few other concepts of a red team I think that every development team should have some exposure to outside of the traditional cybersecurity angle.

id="someone-to-look-for-dicks">Someone to look for dicks

Once upon a time, GitHub was very excitedly looking forward to shipping our FIRST BILLBOARD! It's an odd experience, turning into one of those startups who advertise on 101. Some sort of weird fucked-up sign of maturity and/or sufficient VC dollars. Particularly for a company whose product only exists In The Cloud... seeing real-world analogues is very surreal.

So Marketing worked on it, ran it through design, chatted on some plans as to what it should look like. If I recall, the GitHub thread on it was a couple weeks old, and Cameron McEfee put out a final "I'm going to send this to the printers at the end of the day, so speak now or forever hold your peace!" Some few-dozen people had seen it at this point so it probably was fine.

Anyway, I looked at the last iteration around the same time as Rick Bradley did and we each were like... uh-oh that looks like goatse. Are we sure we aren't goatse-ing all of San Francisco traffic for a few weeks? Seems rude.

Cameron dropped a "holy shit" and got it updated it prior to it hitting the printers. He also put a kind of "dick check" in the general design shipping process for large launches at GitHub- check for various genitals, meme-ability, and really any sort of ways the new work could be used in unintended ways.

I mean, the last thing you want is to have teammates work on something for months and users end up ignoring all their hard work because the new logo looks like a booty or something .

It sounds totally goofy, but it's not a bad idea to have someone in an antagonistic mindset to make sure you're not shipping something awkward, insulting, or inappropriate through your visuals.

Finally, internet shitposters have a valid business use case.

id="someone-with-an-ad-blocker">Someone with an ad blocker

Ad blockers can be somewhat contentious: on one hand it's good to support websites whose access might be free-of-charge; on the other hand, so many of these websites are fucking terrible, with ads and popups and unclosable interstitials.

But some of your users are going to use ad blockers; there's no way around that. So have some asshole on your team constantly piping up if you break navigation with various ad blockers turned on. Yeah, there's some political aspects here — who blocks the blockers? — but every time a site inexplicably doesn't work because someone made it so a file include or piece of HTML wrecks your entire site is one of the most rage-inducing aspects of modern sites out there, particularly if there aren't any ads on the site in the first place.

id="someone-with-a-password-manager">Someone with a password manager

Look: I have a lot to say about sessions and signing in to a product , but suffice to say: there will be password managers for the foreseeable future and holy shit how do you all get the simplest sign-in form so wrong all the time?

Like, at its basic form it's just a username and password. I get — sort of — layering on all this other shit like magic links and 2FA and enterprise sign-in, but so many dev teams don't even get the basic form right: they do something custom in a hair-brained way so that 1Password or other password managers don't auto-fill the form. (Yes, some of that is because 1Password itself has turned into some sort of deranged software that breaks if you look at it, but you get the idea.)

So someone on your team should use a password manager. I mean, you all should, of course, but for the love of god at least get one person on the team to pipe up with "why doesn't my auto-fill work on our site?" And then fix it.

None of these are like, major blockers: people will work around broken forms or websites, and drivers will drive by your phallus on the highway. But they're pretty easy to prevent. The main problem is that when you're building a new feature you have so many other things to worry about... which is why having a kind of "red team" can be so helpful, to come at it with fresh, antagonistic eyes.

Anyway, just wanted you to think about this as you build your products! If you think it's helpful, I'm just about to send these stickers to the printer- let me know if you're interested in grabbing one.

It sounds totally goofy, but it's not a bad idea to have someone in an antagonistic mindset to make sure you're not shipping something awkward, insulting, or inappropriate through your visuals.

Zach Holman

}

{

A one-line Kubernetes fix that saved 600 hours a year

Braxton Schafer · Cloudflare Blog

Every time we restarted Atlantis, the tool we use to plan and apply Terraform changes, we'd be stuck for 30 minutes waiting for it to come back up. No plans, no applies, no infrastructure changes for any repository managed by Atlantis. With roughly 100 restarts a month for credential rotations and onboarding, that added up to over 50 hours of blocked engineering time every month , and paged the on-call engineer every time.

This was ultimately caused by a safe default in Kubernetes that had silently become a bottleneck as the persistent volume used by Atlantis grew to millions of files. Here's how we tracked it down and fixed it with a one-line change.

Mysteriously slow restarts

We manage dozens of Terraform projects with GitLab merge requests (MRs) using Atlantis , which handles planning and applying. It enforces locking to ensure that only one MR can modify a project at a time.

It runs on Kubernetes as a singleton StatefulSet and relies on a Kubernetes PersistentVolume (PV) to keep track of repository state on disk. Whenever a Terraform project needs to be onboarded or offboarded, or credentials used by Terraform are updated, we have to restart Atlantis to pick up those changes — a process that can take 30 minutes.

The slow restart was apparent when we recently ran out of inodes on the persistent storage used by Atlantis, forcing us to restart it to resize the volume. Inodes are consumed by each file and directory entry on disk, and the number available to a filesystem is determined by parameters passed when creating it. The Ceph persistent storage implementation provided by our Kubernetes platform does not expose a way to pass flags to mkfs , so we're at the mercy of default values: growing the filesystem is the only way to grow available inodes, and restarting a PV requires a pod restart.

We talked about extending the alert window, but that would just mask the problem and delay our response to actual issues. Instead, we decided to investigate exactly why it was taking so long.

Bad behavior

When we were asked to do a rolling restart of Atlantis to pick up a change to the secrets it uses, we would run kubectl rollout restart statefulset atlantis , which would gracefully terminate the existing Atlantis pod before spinning up a new one. The new pod would appear almost immediately, but looking at it would show:

$ kubectl get pod atlantis-0 atlantis-0 0/1 Init:0/1 0 30m

...so what gives? Naturally, the first thing to check would be events for that pod. It's waiting around for an init container to run, so maybe the pod events would illuminate why?

$ kubectl events –for=pod/atlantis-0 LAST SEEN TYPE REASON OBJECT MESSAGE 30m Normal Killing Pod/atlantis-0 Stopping container atlantis-server 30m Normal Scheduled Pod/atlantis-0 Successfully assigned atlantis/atlantis-0 to 36com1167.cfops.net 22s Normal Pulling Pod/atlantis-0 Pulling image "oci.example.com/git-sync/master:v4.1.0" 22s Normal Pulled Pod/atlantis-0 Successfully pulled image "oci.example.com/git-sync/master:v4.1.0" in 632ms (632ms including waiting). Image size: 58518579 bytes.

That looks almost normal... but what's taking so long between scheduling the pod and actually starting to pull the image for the init container? Unfortunately that was all the data we had to go on from Kubernetes itself. But surely there had to be something more that can tell us why it's taking so long to actually start running the pod.

Going deeper

In Kubernetes, a component called kubelet that runs on each node is responsible for coordinating pod creation, mounting persistent volumes, and many other things. From my time on our Kubernetes team, I know that kubelet runs as a systemd service and so its logs should be available to us in Kibana. Since the pod has been scheduled, we know the host name we're interested in, and the log messages from kubelet include the associated object, so we could filter for atlantis to narrow down the log messages to anything we found interesting.

We were able to observe the Atlantis PV being mounted shortly after the pod was scheduled. We also observed all the secret volumes mount without issue. However, there was still a big unexplained gap in the logs. We saw:

[operation_generator.go:664] "MountVolume. MountDevice succeeded for volume \"pvc-94b75052-8d70-4c67-993a-9238613f3b99\" (UniqueName: \"kubernetes.io/csi/rook-ceph-nvme.rbd.csi.ceph.com^0001-000e-rook-ceph-nvme-0000000000000002-a6163184-670f-422b-a135-a1246dba4695\") pod \"atlantis-0\" (UID: \"83089f13-2d9b-46ed-a4d3-cba885f9f48a\") device mount path \"/state/var/lib/kubelet/plugins/kubernetes.io/csi/rook-ceph-nvme.rbd.csi.ceph.com/d42dcb508f87fa241a49c4f589c03 globalmount\"" pod="atlantis/atlantis-0"

[pod_workers.go:1298] "Error syncing pod, skipping" err="unmounted volumes=[atlantis-storage], unattached volumes=[], failed to process volumes=[]: context deadline exceeded" pod="atlantis/atlantis-0" podUID="83089f13-2d9b-46ed-a4d3-cba885f9f48a"

[util.go:30] "No sandbox for pod can be found. Need to start a new one" pod="atlantis/atlantis-0"

The last two messages looped several times until eventually we observed the pod actually start up properly.

So kubelet thinks that the pod is otherwise ready to go, but it's not starting it and something's timing out.

```
}
{
```

You Are the Low-Hanging Fruit

Yegor Bugayenko

Let's say, you are a startup founder, like myself . Try to hire a sales guy. Offer him a commission-only payment scheme. Listen to his reaction: he will demand that you pay a fixed salary too, on top of commission. Try to convince him that commission-only is a more reasonable and motivating setup. Goto 1. After a number of iterations you realize that the mission is impossible. Sales people are good at selling and the best thing they sell is the idea that their time must be compensated. Even if they don't sell the product to your customers. If you don't buy the idea, they go find another loser who will. Something similar happens when you try to pay programmers by result. They easily convince you to pay for their time . And you do.

City of God (2002) by Kátia Lund

A sales rep doesn't know how to write code. Most of them don't even know how computers work. However, he perfectly knows how to bullshit people. That's exactly why we need him. Because we don't know how to bullshit people and we don't want to learn it.

Now, two strategies lie in front of him. He can use the skill against the prospects on the market and turn them into paying customers. He can also use the same selling skill against you, the owner of the startup. Instead of selling the product to the market he can sell himself to you. He can sell the idea that even if he fails to sell the product to the customers he still deserves a decent weekly paycheck.

What do you think, which sale is easier to make? What would you do in his shoes? The answer is obvious. The customers are far away and they have no mercy. If they don't like the offer, they simply hang up on him and that's it. You, on the other hand, sit next to him in the same office and can't hang up. You are the low-hanging fruit . You are the weakest prey he can reach out to.

In order to make him focus on external obstacles, you should make internal ones harder to overcome.

A customer is an external obstacle that he must overcome in order to get paid, as a sales commission. You are an internal obstacle, which he may also overcome to get a fixed weekly payment. You need him to fight the external obstacle. However, he is free to choose the easiest path.

In order to make him focus on external obstacles, you should make internal ones harder to overcome. Joseph Stalin once said that "in the Soviet army it takes more courage to retreat than advance." This war-time concept seems relevant to the sales guys you hire. It must be harder for them to talk you into paying them a fixed salary than to convince a prospect to buy your product.

Emotionally, for you it may be rather challenging to constantly push him back. Just like it's often hard to say "No" to a vagrant begging for a dollar at the corner. Beggars, unless they are physically disabled, also, just like sales people, have two possible life strategies. Either find a job or beg at the corner. The begging strategy, for the vagrant and for the sales rep, is easier to pursue.

Programmers are not much different. They also have two strategies. They can solve technical problems by merging qualified pull requests. They can also persuade you that you must pay for their time, not their pull requests. Which obstacle is going to be harder for them to overcome depends on you .

First, you set up a formula for measuring their contribution. Second, you bind their paychecks to it: they get paid not by you , but by merged pull requests . Finally, you taboo the very possibility of discussing time-based compensation.

Taboo the very possibility of discussing time-based compensation.

What you get is a technical team focused on resolving external problems. The team will advance because it will be pointless to retreat . You simply won't pay them for their time. No matter how many times they repeat "I was working hard the entire weekend."

Incentives shape behavior. If you reward excuses, you buy excuses. If you reward results, you get results. As a founder, your job is to eliminate the temptation for your team to sell you anything other than tangible artifacts.

```
}
{
```

Electric Geek Transportation Systems

Jeff Atwood · Coding Horror (Jeff Atwood)

I've never thought of myself as a "car person." The last new car I bought (and in fact, now that I think about it, the first new car I ever bought) was the quirky 1998 Ford Contour SVT . Since then, we bought a VW station wagon in 2011 and a Honda minivan in 2012 for family transportation duties. That's it. Not exactly the stuff The Stig's dreams are made of.

The station wagon made sense for a family of three, but became something of a disappointment because it was purchased before — surprise! — we had twins . As Mark Twain once said :

Sufficient unto the day is one baby. As long as you are in your right mind don't you ever pray for twins. Twins amount to a permanent riot. And there ain't any real difference between triplets and an insurrection.

I'm here to tell you that a station wagon doesn't quite cut it as a permanent riot abatement tool. For that you need a full sized minivan.

I'm with Philip Greenspun . Like black socks and sandals, minivans are actually... kind of awesome? Don't believe all the SUV propaganda. Minivans are flat out superior vehicle command centers. Swagger wagons , really.

The A-Team drove a van , not a freakin' SUV. I rest my case.

After 7 years, the station wagon had to go. We initially looked at hybrids because, well, isn't that required in California at this point ? But if you know me at all, you know I'm a boil the sea kinda guy at heart. I figure if you're going to flirt with partially electric cars, why not put aside these half measures and go all the way?

Do you remember that rapturous 2014 Oatmeal comic about the Tesla model S? Even for a person who has basically zero interest in automobiles, it did sound really cool .

It's been 5 years, but from time to time I'd see some electric vehicle on the road and I'd think about that Intergalactic SpaceBoat of Light and Wonder . Maybe it's time for our family to jump on the electric car trend, too, and just late enough that we can avoid the bleeding edge and end up merely on the... leading edge?

That's why we're now the proud owners of a fully electric 2019 Kia Niro .

I've somehow gone from being a person who basically doesn't care about cars at all... to being one of those insufferable electric car people who won't shut up about them . I apologize in advance. If you suddenly feel an overwhelming urge to close this browser tab, I don't blame you.

I was expecting another car, like the three we bought before. What I got, instead, was a transformation:

Yes, yes, electric cars are clean, but it's a revelation how clean everything is in an electric. You take for granted how dirty and noisy gas based cars are in daily operation – the engine noise, the exhaust fumes, the brake dust on the rims, the oily residues and thin black film that descends on everything, the way you have to wash your hands every time you use the gas station pumps. You don't fully appreciate how oppressive those little dirty details were until they're gone.

Electric cars are (almost) completely silent. I guess technically in 2019 electric cars require artificial soundmakers at low speed for safety, and this car has one. But The Oatmeal was right. Electric cars feel like spacecraft because they move so effortlessly. There's virtually no delay from action to reaction, near immediate acceleration and deceleration... with almost no sound or vibration at all, like you're in freakin' space! It's so immensely satisfying!

Electric cars aren't just electric , they're utterly digital to their very core. Gas cars always felt like the classic 1950s Pixar Cars world of grease monkeys and machine shop guys, maybe with a few digital bobbins added here and there as an afterthought. This electric car, on the other hand, is squarely in the post-iPhone world of everyday digital gadgets. It feels more like a giant smartphone than a car. I am a programmer, I'm a digital guy, I love digital stuff. And electric cars are part of my world, rather than the other way around. It feels good.

Electric cars are mechanically much simpler than gasoline cars, which means they are inherently more reliable and cheaper to maintain. An internal combustion engine has hundreds of moving parts, many of which require regular maintenance, fluids, filters, and tune ups. It also has a complex transmission to translate the narrow power band of a gas powered engine. None of this is necessary on an electric vehicle, whose electric motor is basically one moving part with simple 100% direct drive from the motor to the wheels. This newfound simplicity is deeply appealing to a guy who always saw cars as incredibly complicated (but computers, not so much).

Being able to charge at home overnight is perhaps the most radical transformation of all. Your house is now a "gas station." Our Kia Niro has a range of about 250 miles on a full battery. With any modern electric car, provided you drive less than 200 miles a day round trip (who even drives this much?), it's very unlikely you'll ever need to "fill the tank" anywhere but at home. Ever. It's so strange to think that in 50 years, gas stations may eventually be as odd to see in public as public telephone booths now are. Our charger is, conveniently enough, right next to the driveway since that's where the power breaker box was. With the level 2 charger installed, it literally looks like a gas pump on the side of the house, except this one "pumps"... electrons.

This electric car is such a great experience. It's so much better than our gas powered station wagon that I swear, if there was a fully electric minivan (there isn't) I would literally sell our Honda minivan tomorrow and switch over. Without question. And believe me, I had no plans to sell

```
}
{
```

GTX 1080 Ti for Local LLM

Ariya Hidayat

Despite being over eight years old, the NVIDIA GTX 1080 Ti remains a compelling choice for enthusiasts keen on running LLM locally.

Initially launched in early 2017 with a $699 MSRP, this GTX 1080 Ti card quickly earned a “legendary GPU” reputation among tech reviewers and YouTubers. Today, it’s readily available on the second-hand market (particularly in Northern California or on eBay) for around $150, often even less if you’re lucky.

For this modest price, you acquire a card with 11 GB of VRAM, an unconventional yet highly practical configuration for modern LLMs. With quantized models, 11 GB typically offers ample space for model weights and the context windows crucial for RAG workflows, coding assistants, and other use cases. This makes it an ideal sweet spot for hobbyists: affordable enough for experimentation, yet powerful enough to handle practical workloads.

Please note that local LLM inference performance is primarily assessed by two metrics:

Prompt Processing: How quickly the LLM processes the input context.

Token Generation: How rapidly the LLM produces output (the “answer”).

These metrics, often denoted as pp512 and tg128 (where the numbers represent total tokens), can be measured using llama.cpp , a very popular inference engine. As mentioned in our previous article on running LLMs locally with LM Studio and Jan, llama.cpp is one of the engines powering these applications.

For LLM tasks requiring long context windows, such as RAG or coding assistant, pp512 performance is paramount. Slow pp512 leads to noticeable lag as large prompts take seconds to preprocess before any output appears. Meanwhile, for dynamic chats or creative writing, tg128 is more critical, as it dictates the fluidity of the model’s responses. llama.cpp with CUDA

To run benchmarks, you’ll first need a CUDA-enabled build of llama.cpp , which involves installing NVIDIA’s CUDA toolkit and ensuring your development tools are correctly set up. A quick sanity check involves verifying the versions of nvcc, CMake, and gcc:

nvcc –version cmake –version gcc –version

If all three commands return version numbers, you’re ready to proceed.

Next, clone the llama.cpp repository and build it with CUDA enabled:

git clone https://github.com/ggml-org/llama.cpp cd llama.cpp cmake -B build -DGGML_CUDA=ON cmake –build build –config Release

This compilation process can take some time, so be prepared for a short wait.

Once built, test your setup with a small model like Google’s Gemma-3 1B (approximately 800 MB). You can interact with it via the command line (llama-cli) or launch a lightweight web UI:

./build/bin/llama-server

Running nvtop concurrently will confirm that the GPU is actively utilized during inference.

With llama.cpp configured, benchmarking is straightforward using the included llama-bench tool. For consistent comparisons, Llama-2 7B Q4_0 quantization is a popular choice, being small enough to run comfortably and widely benchmarked.

Execute the following command:

./build/bin/llama-bench -ngl 100 -m llama-2-7b. Q4_0.gguf

The output will provide both pp512 and tg128. While these speeds may not match modern GPUs, they are more than enough for local experimentation. Crucially, the pp512 performance remains competitive enough to make retrieval-heavy workloads (like document-based question answering) viable. The GTX 1080 Ti may not be the fastest, but it offers an excellent entry point into local AI.

Community benchmarks in llama.cpp discussions, illustrate how this GPU compares:

CUDA (NVIDIA GPUs): https://github.com/ggml-org/llama.cpp/discussions/15013

Metal (Apple Silicon): https://github.com/ggml-org/llama.cpp/discussions/4167

ROCm (AMD GPUs): https://github.com/ggml-org/llama.cpp/discussions/15021

Vulkan (cross-platform): https://github.com/ggml-org/llama.cpp/discussions/10879

These threads are invaluable resources for real-world performance data, aiding decisions on building new local inference rigs or repurposing existing hardware.

A sample of these results:

Apple M4 Pro: pp512 = 439 tok/s and tg128 = 50 tok/s

GTX 1080 Ti: pp512 = 1084 tok/s and tg128 = 62 tok/s

RX 9060 XT: pp512 = 1478 tok/s and tg128 = 65 tok/s

For consistent comparisons, Llama-2 7B Q4_0 quantization is a popular choice, being small enough to run comfortably and widely benchmarked.

Ariya Hidayat

}

{

Stay Gold, America

Jeff Atwood · Coding Horror (Jeff Atwood)

We are at an unprecedented point in American history, and I'm concerned we may lose sight of the American Dream :

The costs of housing, healthcare, and education have soared far beyond the pace of inflation and wage growth .

We are a democracy, but 144 million Americans – 42% of the adults who live here – do not vote and have no say in what happens.

Wealth concentration has reached historic levels . The top 1% of households control 32% of all wealth, while the bottom 50% only have 2.6%.

We must act now to keep the dream alive. Our family made eight $1 million donations to nonprofit groups working to support those most currently in need:

Team Rubicon – Mobilizing veterans to continue their service, leveraging their skills and experience to help Americans prepare, respond, and recover from natural disasters.

Children's Hunger Fund – Provides resources to local churches in the United States and around the world to meet the needs of impoverished community members.

PEN America – Defends writers against censorship and abuse, supports writers in need of emergency assistance, and amplifies the writing of incarcerated prisoners. (One of my personal favorites; I've seen the power of writing transform our world many times.)

The Trevor Project – Working to change hearts, minds, and laws to support the lives of young adults seeking acceptance as fellow Americans.

NAACP Legal Defense and Educational Fund – Legal organization with a historic record of advancing racial justice and reducing inequality.

First Generation Investors – Introduces high school students in low-income areas to the fundamentals of investing, providing them real money to invest, encouraging long-term wealth accumulation and financial literacy among underserved youth.

Global Refuge – Supporting migrants and refugees from around the globe, in partnership with community-based legal and social service providers nationwide, helping rebuild lives in America.

Planned Parenthood – Provides essential healthcare services and resources that help individuals and families lead healthier lives.

I encourage every American to contribute soon, however you can, to organizations you feel are effectively helping those most currently in need here in America.

We must also work toward deeper changes that will take decades to achieve. Over the next five years, my family pledges half our remaining wealth towards long term efforts ensuring that all Americans continue to have access to the American Dream.

I never thought my family would be able to do this. My parents are of hardscrabble rural West Virginia and rural North Carolina origins. They barely managed to claw their way to the bottom of the middle class by the time they ended up in Virginia. Unfortunately, due to the demons passed on to them by their parents, my father was an alcoholic and my mother participated in the drinking. She ended up divorcing my father when I was 16 years old. It was only after the divorce that my parents were able to heal themselves, heal their only child, and stop the drinking, which was so destructive to our family. If the divorce hadn't forced the issue, alcohol would have inevitably destroyed us all.

My parents may not have done everything right, but they both unconditionally loved me. They taught me how to fully, deeply receive love, and the profound joy of reflecting that love upon everyone around you.

I went on to attend public school in Chesterfield County, Virginia. In 1992 I graduated from the University of Virginia, founded by Thomas Jefferson .

During college, I worked at Safeway as a part-time cashier, earning the federal minimum wage , scraping together whatever money I could through government Pell grants, scholarships, and other part-time work to pay my college tuition. Even with lower in-state tuition , it was rocky. Sometimes I could barely manage tuition payments. And that was in 1992, when tuition was only $3,000 per year. It is now $23,000 per year. College tuition at a state school increased by 8 times over the last 30 years. These huge cost increases for healthcare, education, and housing are not compatible with the American Dream.

Programmers all over the world helped make an American Dream happen in 2008 when we built Stack Overflow , a Q&A website for programmers creating a shared Creative Commons knowledge base for the world. We did it democratically, because that's the American way . We voted to rank questions and answers, and held elections for community moderators using ranked choice voting . We built a digital democracy – of the programmers, by the programmers, for the programmers. It worked .

With the guidance of my co-founder Joel Spolsky , I came to understand that the digital democracy of Stack Overflow was not enough. We must be brave enough to actively, openly share love with each other. That became the foundation for Discourse , a free, open source tool for constructive, empathetic community discussions that are also Creative Commons. We can disagree in those discussions because Discourse empowers communities to set boundaries the community agrees on , providing tools to democratically govern and strongly moderate by enforcing these boundaries. Digital democracy and empathy , for everyone.

```
}
{
```

Some Things Just Take Time

Armin Ronacher · Armin Ronacher (Lucumr)

Trees take quite a while to grow. If someone 50 years ago planted a row of oaks or a chestnut tree on your plot of land, you have something that no amount of money or effort can replicate. The only way is to wait. Tree-lined roads, old gardens, houses sheltered by decades of canopy: if you want to start fresh on an empty plot, you will not be able to get that.

Because some things just take time.

We know this intuitively. We pay premiums for Swiss watches, Hermès bags and old properties precisely because of the time embedded in them. Either because of the time it took to build them or because of their age. We require age minimums for driving, voting, and drinking because we believe maturity only comes through lived experience.

Yet right now we also live in a time of instant gratification, and it's entering how we build software and companies. As much as we can speed up code generation, the real defining element of a successful company or an Open Source project will continue to be tenacity. The ability of leadership or the maintainers to stick to a problem for years, to build relationships, to work through challenges fundamentally defined by human lifetimes.

The current generation of startup founders and programmers is obsessed with speed. Fast iteration, rapid deployment, doing everything as quickly as possible. For many things, that's fine. You can go fast, leave some quality on the table, and learn something along the way.

But there are things where speed is actively harmful, where the friction exists for a reason. Compliance is one of those cases. There's a strong desire to eliminate everything that processes like SOC2 require, and an entire industry of turnkey solutions has sprung up to help — Delve just being one example, there are more.

There's a feeling that all the things that create friction in your life should be automated away. That human involvement should be replaced by AI-based decision-making. Because it is the friction of the process that is the problem. When in fact many times the friction, or that things just take time, is precisely the point.

There's a reason we have cooling-off periods for some important decisions in one's life. We recognize that people need time to think about what they're doing, and that doing something right once doesn't mean much because you need to be able to do it over a longer period of time.

Vibe Slop At Inference Speeds

AI writes code fast which isn't news anymore. What's interesting is that we're pushing this force downstream: we seemingly have this desire to ship faster than ever, to run more experiments and that creates a new desire, one to remove all the remaining friction of reviews, designing and configuring infrastructure, anything that slows the pipeline. If the machines are so great, why do we even need checklists or permission systems? Express desire, enjoy result.

Because we now believe it is important for us to just do everything faster. But increasingly, I also feel like this means that the shelf life of much of the software being created today — software that people and businesses should depend on — can be measured only in months rather than decades, and the relationships alongside.

In one of last year's earlier YC batches, there was already a handful that just disappeared without even saying what they learned or saying goodbye to their customers. They just shut down their public presence and moved on to other things. And to me, that is not a sign of healthy iteration. That is a sign of breaking the basic trust you need to build a relationship with customers. A proper shutdown takes time and effort, and our current environment treats that as time not wisely spent. Better to just move on to the next thing.

This is extending to Open Source projects as well. All of a sudden, everything is an Open Source project, but many of them only have commits for a week or so, and then they go away because the motivation of the creator already waned. And in the name of experimentation, that is all good and well, but what makes a good Open Source project is that you think and truly believe that the person that created it is either going to stick with it for a very long period of time, or they are able to set up a strategy for succession, or they have created enough of a community that these projects will stand the test of time in one form or another.

Relatedly, I'm also increasingly skeptical of anyone who sells me something that supposedly saves my time. When all that I see is that everybody who is like me, fully onboarded into AI and agentic tools, seemingly has less and less time available because we fall into a trap where we're immediately filling it with more things.

We all sell each other the idea that we're going to save time, but that is not what's happening. Any time saved gets immediately captured by competition. Someone who actually takes a breath is outmaneuvered by someone who fills every freed-up hour with new output. There is no easy way to bank the time and it just disappears.

I feel this acutely. I'm very close to the red-hot center of where economic activity around AI is taking place, and more than anything, I have less and less time, even when I try to purposefully scale back and create the space. For me this is a problem. It's a problem because even with the best intentions, I actually find it very hard to create quality when we are quickly commoditizing software, and the machines make it so appealing.

I keep coming back to the trees. I've been maintaining Open Source projects for close to two decades now. The last startup I worked on, I spent 10 years at. That's not because

```
}
{
```

Agent Psychosis: Are We Going Insane?

Armin Ronacher · Armin Ronacher (Lucumr)

You can use Polecats without the Refinery and even without the Witness or Deacon. Just tell the Mayor to shut down the rig and sling work to the polecats with the message that they are to merge to main directly. Or the polecats can submit MRs and then the Mayor can merge them manually. It's really up to you. The Refineries are useful if you have done a LOT of up-front specification work, and you have huge piles of Beads to churn through with long convoys.

— Gas Town Emergency User Manual , Steve Yegge

Many of us got hit by the agent coding addiction. It feels good, we barely sleep, we build amazing things. Every once in a while that interaction involves other humans, and all of a sudden we get a reality check that maybe we overdid it. The most obvious example of this is the massive degradation of quality of issue reports and pull requests. As a maintainer many PRs now look like an insult to one's time, but when one pushes back, the other person does not see what they did wrong. They thought they helped and contributed and get agitated when you close it down.

But it's way worse than that. I see people develop parasocial relationships with their AIs, get heavily addicted to it, and create communities where people reinforce highly unhealthy behavior. How did we get here and what does it do to us?

I will preface this post by saying that I don't want to call anyone out in particular, and I think I sometimes feel tendencies that I see as negative, in myself as well. I too, have thrown some vibeslop up to other people's repositories.

Our Little Dæmons

In His Dark Materials, every human has a dæmon, a companion that is an

externally visible manifestation of their soul. It lives alongside as an

animal, but it talks, thinks and acts independently. I'm starting to relate our

relationship with agents that have memory to those little creatures. We become

dependent on them, and separation from them is painful and takes away from our

new-found identity. We're relying on these little companions to validate us and

to collaborate with. But it's not a genuine collaboration like between humans,

it's one that is completely driven by us, and the AI is just there for the ride.

We can trick it to reinforce our ideas and impulses. And we act through this

AI. Some people who have not programmed before, now wield tremendous powers,

but all those powers are gone when their subscription hits a rate limit and

their little dæmon goes to sleep.

Then, when we throw up a PR or issue to someone else, that contribution is the

result of this pseudo-collaboration with the machine. When I see an AI pull

request come in, or on another repository, I cannot tell how someone created it,

but I can usually after a while tell when it was prompted in a way that is

fundamentally different from how I do it. Yet it takes me minutes to figure

this out. I have seen some coding sessions from others and it's often done with

clarity, but using slang that someone has come up with and most of all: by

completely forcing the AI down a path without any real critical thinking.

Particularly when you're not familiar with how the systems are supposed to work,

giving in to what the machine says and then thinking one understands what is

going on creates some really bizarre outcomes at times.

But people create these weird relationships with their AI agent and once you see how some prompt their machines, you realize that it dramatically alters what comes out of it. To get good results you need to provide context, you need to make the tradeoffs, you need to use your knowledge. It's not just a question of using the context badly, it's also the way in which people interact with the machine. Sometimes it's unclear instructions, sometimes it's weird role-playing and slang, sometimes it's just swearing and forcing the machine, sometimes it's a weird ritualistic behavior. Some people just really ram the agent straight towards the most narrow of all paths towards a badly defined goal with little concern about the health of the codebase.

Addicted to Prompts

These dæmon relationships change not just how we work, but what we produce. You can completely give in and let the little dæmon run circles around you. You can reinforce it to

```
}
{
```

There is no longer any such thing as Computer Security

Jeff Atwood · Coding Horror (Jeff Atwood)

Remember “cybersecurity”?

Mysterious hooded computer guys doing mysterious hooded computer guy... things! Who knows what kind of naughty digital mischief they might be up to?

Unfortunately, we now live in a world where this kind of digital mischief is literally rewriting the world’s history. For proof of that, you need look no further than this single email that was sent March 19th, 2016.

If you don’t recognize what this is, it is a phishing email .

This is by now a very, very famous phishing email, arguably the most famous of all time. But let’s consider how this email even got sent to its target in the first place:

An attacker slurped up lists of any public emails of 2008 political campaign staffers.

One 2008 staffer was also hired for the 2016 political campaign.

That particular staffer had non-public campaign emails in their address book, and one of them was a powerful key campaign member with an extensive email history.

On successful phish leads to an even wider address book attack net down the line. Once they gain access to a person’s inbox, they use it to prepare to their next attack. They’ll harvest existing email addresses, subject lines, content, and attachments to construct plausible looking boobytrapped emails and mail them to all of their contacts. How sophisticated and targeted to a particular person this effort is determines whether it’s so-called “spear” phishing or not.

In this case is it was not at all targeted. This is a remarkably unsophisticated, absolutely generic routine phishing attack. There is zero focused attack effort on display here. But note the target did not immediately click the link in the email!

Instead, he did exactly what you’d want a person to do in this scenario: he emailed IT support and asked if this email was valid. But IT made a fatal mistake in their response .

Do you see it? Here’s the kicker:

Mr. Delavan, in an interview, said that his bad advice was a result of a typo: He knew this was a phishing attack, as the campaign was getting dozens of them. He said he had meant to type that it was an “illegitimate” email, an error that he said has plagued him ever since.

One word. He got one word wrong. But what a word to get wrong, and in the first sentence! The email did provide the proper Google address to reset your password. But the lede was already buried since the first sentence said “legitimate;” the phishing link in that email was then clicked. And the rest is literally history.

What’s even funnier (well, in the way of gallows humor, I guess) is that public stats were left enabled for that bit.ly tracking link, so you can see exactly what crazy domain that “Google login page” resolved to, and that it was clicked exactly twice, on the same day it was mailed.

As I said, these were not exactly sophisticated attackers. So yeah, in theory an attentive user could pay attention to the browser’s address bar and notice that after clicking the link, they arrived at

http://myaccount.google.com-securitysettingpage.tk/security/signinoptions/password

instead of

https://myaccount.google.com/security

Note that the phishing URL is carefully constructed so the most “correct” part is at the front, and weirdness is sandwiched in the middle. Unless you’re paying very close attention and your address bar is long enough to expose the full URL, it’s... tricky. See this 10 second video for a dramatic example.

1×

(And if you think that one’s good, check out this one. Don’t forget all the Unicode look-alike trickery you can pull, too.)

I originally wrote this post as a presentation for the Berkeley Computer Science Club back in March, and at that time I gathered a list of public phishing pages I found on the web.

nightlifesofl.com ehizaza-limited.com tcgoogle.com appsgoogie.com security-facabook.com

Of those five examples from 6 months ago, one is completely gone, one loads just fine, and three present an appropriately scary red interstitial warning page that strongly advises you not to visit the page you’re trying to visit, courtesy of Google’s safe browsing API . But of course this kind of shared blacklist domain name protection will be completely useless on any fresh phishing site. (Don’t even get me started on how blacklists have never really worked anyway.)

It doesn’t exactly require a PhD degree in computer science to phish someone:

Buy a crazy long, realistic looking domain name.

Point it to a cloud server somewhere.

Get a free HTTPS certificate courtesy of our friends at Let’s Encrypt .

Build a realistic copy of a login page that silently transmits everything you type in those login fields to you – perhaps

}

standard-article(title: [Appending text to Obsidan via Shortcuts], author: [Rob], source-name: [Rob Allen (akrabat)], images: (), paragraphs: ([Sometimes it's helpful to add some text to my current Obsidian daily note without having to switch to Obsidian, find the daily note and then type my text.], [To do this, we can use the magic of Obsidian's obsidian:/\/ URI schema and automate the text capture in Apple Shortcuts, with an assigned keyboard shortcut to activate it.], [This is the Shortcut:], [There's a number of steps in the Shortcut. Firstly we Ask For some text which will display a dialog box for us to type in. Then we Replace any trailing whitespace, add then use a Text action to prepend the current time as a third level heading. This is the text to be added, so we URL Encode it and use another Text action to create the full obsidian:/\/ URI to be Open ed.], [The URI is:], [obsidian://daily?vault=General&append=1&content={URL Encoded Text}], [The daily endpoint only works if the Daily notes core plugin is enabled. We then need to specify which vault we're using. The content is the URL encoded text to be appended and the append parameter tells obsidian to place the text at the end of the note.], [I've assigned it to ^⌥⌘= and pressing that key combination runs it for me.], [Now I have a simple way to add some text to the current note without interrupting my context.], [If this shortcut would be useful to you, you can download it here .],), insert-map: (:), word-count: 235, edited-for-length: false, debug-mode: false,)

{

ANALYSIS

standard-article(title: [github.com/garyburd/redigo has been deleted], author: [kevin], source-name: [Kevin Burke], images: (), paragraphs: ([One of the first ever Redis libraries for Go was hosted at github.com/garyburd/redigo.], [It has been deprecated for some time and has now been finally removed altogether from Github. If you still have a dependency on this project, this means that will be broken now.], [github.com/gomodule/redigo should be a drop-in replacement for github.com/garyburd/redigo. If you still depend on the old service, update the organization name, and you should be good to go.], [My hope in writing this is to save time for the next person who is searching for a replacement for this library. Please pay it forward by taking a tiny bit of extra time the next time something breaks, and tell people how to fix the problem.],), insert-map: (:), word-count: 119, edited-for-length: false, debug-mode: false,)

The Fall of JavaScript

Yegor Bugayenko

In 1995, Brendan Eich was hired by Netscape and asked to create a language for their HTML browser. Rumors say, he designed Mocha in 10 days , later renamed to LiveScript , and then to JavaScript . It was planned to make it similar to Scheme , a LISP-syntax language. Instead, to please the crowd of C++/Java coders, it was made syntactically similar to Java. In 2008, Brendan made a tragic mistake: he donated $1,000 in support of Californian anti-gay marriage law . In 2014, he joined Mozilla as a CEO and the crowd remembered his anti-diversity gesture. He had to step down and founded Brave Software, the developer of the Brave browser. Somewhere around that time they started to kill JavaScript. Still doing it pretty good, thanks to recent ECMAScript updates and TypeScript .

In the JavaScript created 30 years ago, objects were primitive associative arrays of properties. Either data, a function, or another object may be attached to a property of an object. Let's see what Brendan said about JS in 2008:

I'm happy that I chose Scheme-ish first-class functions and Self-ish prototypes as the main ingredients.

First-class functions mean that it's possible to use a function as a value assignable to a variable. And then he concluded (pay attention, it's C not C++):

Yet many curse it, including me. I still think of it as a quickie love-child of C and Self.

Self , which he refers to a few times, is a prototype-based object-oriented language designed by David Ungar and Randall Smith in Xerox PARC , then Stanford University , and then Sun Microsystems . Self doesn't have classes, unlike Java or C++. Instead, it only has objects. To create a new object in Self we make a copy of an existing object, known as prototype, and then modify some of its slots (attributes).

In Self, objects don't have types: all method calls are dispatched in runtime. For example, we ask a book to rename itself:

The rename is the method of the book that we call with a single string argument. The computer doesn't know anything about the book until it's time to call the rename method. Obviously, such a duck typing has its performance drawback. Every rename leads to a search in a virtual table of book . To the contrary, C++, where types are known in compile time, can dispatch rename() instantly:

Types (classes in C++ and interfaces in Java) and type annotations are helpful—to the compiler. To us humans they are a burden. They require us to do the work of the compiler. We have to pollute our code with messages like: "This object b is of type Book , please remember." The compiler must be smart enough to understand it without our hints.

This is a debatable topic though. Some believe that type annotations help programmers better understand the code and make fewer mistakes. I'm also in favor of fewer mistakes, but would rather expect the compiler to infer types automatically, without my annotations. If I do b.rename() and b is known to be a car instead of a book, I would expect the compiler to figure this out on its own and refuse to compile.

Anyway, JavaScript was designed as a prototype-based dynamically typed language with a minimalistic syntax that resembles Java. It worked perfectly fine until the industry decided to "fix" it.

JavaScript worked perfectly fine until the industry decided to fix it.

In 2008, Mozilla and others proposed ECMAScript 4, which included classes, modules, and other features. Microsoft took an extreme position , refusing to accept any part of ES4. Chris Wilson, Microsoft's Internet Explorer platform architect, criticized ES4 for trying to introduce too many changes. Brendan Eich accused Wilson of spreading falsehoods and playing political games. ES4 was abandoned , and classes were dropped.

Then, in 2012, Microsoft created TypeScript , a JavaScript with type annotations and classes. Since classes weren't in the standard, Microsoft made their own.

Finally, in 2015, ECMAScript 6 added classes (among other features) to the JavaScript specification. Many ES4 features, including classes, were revived in a "maximally minimal" form. The crowd of Java/C++ developers got what they wanted.

Let's hear what Douglas Crockford , the creator of JSON and JSLint , said in 2014:

Class-free programming is JavaScript's contribution to humanity.

Unfortunately, not anymore. It looks like the developers of recent versions of JS believe in something else. Or maybe they don't want to make a contribution to humanity anymore. Maybe they just want to make the crowd happy.

Type annotations and classes don't match with the concept of class-free object-based programming of JavaScript. They came from Java or C++ but don't fit in. Some programmers may find them helpful, but only because they are used to seeing them in other languages. JavaScript is not Java, even though the names look similar.

Java, with its classes , implementation inheritance , and static methods , is OOP for dummies. It's object-oriented programming for those who, according to Douglas Crockford , don't know what object-oriented programming is. JavaScript, on the other hand, is a member of conceptually solid OO languages, like Smalltalk and Self. Blending Java flavors into JavaScript only ruins the flavor of the latter.

ES4 was abandoned , and classes were dropped.

Yegor Bugayenko

}

{

AI Security for Apps is now generally available

Liam Reese · Cloudflare Blog

Cloudflare's AI Security for Apps detects and mitigates threats to AI-powered applications. Today, we're announcing that it is generally available.

We're shipping with new capabilities like detection for custom topics, and we're making AI endpoint discovery free for every Cloudflare customer—including those on Free, Pro, and Business plans—to give everyone visibility into where AI is deployed across their Internet-facing apps.

We're also announcing an expanded collaboration with IBM, which has chosen Cloudflare to deliver AI security to its cloud customers. And we're partnering with Wiz to give mutual customers a unified view of their AI security posture.

A new kind of attack surface

Traditional web applications have defined operations: check a bank balance, make a transfer. You can write deterministic rules to secure those interactions.

AI-powered applications and agents are different. They accept natural language and generate unpredictable responses. There's no fixed set of operations to allow or deny, because the inputs and outputs are probabilistic. Attackers can manipulate large language models to take unauthorized actions or leak sensitive data. Prompt injection, sensitive information disclosure, and unbounded consumption are just a few of the risks cataloged in the OWASP Top 10 for LLM Applications .

These risks escalate as AI applications become agents. When an AI gains access to tool calls—processing refunds, modifying accounts, providing discounts, or accessing customer data—a single malicious prompt becomes an immediate security incident.

Customers tell us what they're up against. "Most of Newfold Digital's teams are putting in their own Generative AI safeguards, but everybody is innovating so quickly that there are inevitably going to be some gaps eventually," says Rick Radinger, Principal Systems Architect at Newfold Digital, which operates Bluehost, HostGator, and Domain.com.

What AI Security for Apps does

We built AI Security for Apps to address this. It sits in front of your AI-powered applications, whether you're using a third-party model or hosting your own, as part of Cloudflare's reverse proxy . It helps you (1) discover AI-powered apps across your web property, (2) detect malicious or off-policy behavior to those endpoints, and (3) mitigate threats via the familiar WAF rule builder.

Discovery — now free for everyone

Before you can protect your LLM-powered applications, you need to know where they're being used. We often hear from security teams who don't have a complete picture of AI deployments across their apps, especially as the LLM market evolves and developers swap out models and providers.

AI Security for Apps automatically identifies LLM-powered endpoints across your web properties, regardless of where they're hosted or what the model is. Starting today, this capability is free for every Cloudflare customer, including Free, Pro, and Business plans.

Cloudflare's dashboard page of web assets, showing 2 example endpoints labelled as cf-llm

Discovering these endpoints automatically requires more than matching common path patterns like /chat/completions . Many AI-powered applications don't have a chat interface: think product search, property valuation tools, or recommendation engines. We built a detection system that looks at how endpoints behave , not what they're called. To confidently identify AI-powered endpoints, sufficient valid traffic is required.

AI-powered endpoints that have been discovered will be visible under Security → Web Assets , labeled as cf-llm . For customers on a Free plan, endpoint discovery is initiated when you first navigate to the Discovery page . For customers on a paid plan, discovery occurs automatically in the background on a recurring basis. If your AI-powered endpoints have been discovered, you can review them immediately.

AI Security for Apps detections follow the always-on approach for traffic to your AI-powered endpoints. Each prompt is run through multiple detection modules for prompt injection, PII exposure, and sensitive or toxic topics. The results—whether the prompt was malicious or not—are attached as metadata you can use in custom WAF rules to enforce your policies. We are continuously exploring ways to leverage our global network, which sees traffic from roughly 20% of the web , to identify new attack patterns across millions of sites before they reach yours.

New in GA: Custom topics detection

The product ships with built-in detection for common threats: prompt injections, PII extraction , and toxic topics . But every business has its own definition of what's off-limits. A financial services company might need to detect discussions of specific securities. A healthcare company might need to flag conversations that touch on patient data. A retailer might want to know when customers are asking about competitor products.

The new custom topics feature lets you define these categories. You specify the topic, we inspect the prompt and output a relevance score that you can use to log, block, or handle however you decide. Our goal is to build an extensible tool that flexes to your use cases.

```
}
{
```

Investigating multi-vector attacks in Log Explorer

Jen Sells · Cloudflare Blog

In the world of cybersecurity, a single data point is rarely the whole story. Modern attackers don't just knock on the front door; they probe your APIs, flood your network with "noise" to distract your team, and attempt to slide through applications and servers using stolen credentials.

To stop these multi-vector attacks, you need the full picture. By using Cloudflare Log Explorer to conduct security forensics, you get 360-degree visibility through the integration of 14 new datasets, covering the full surface of Cloudflare's Application Services and Cloudflare One product portfolios. By correlating telemetry from application-layer HTTP requests, network-layer DDoS and Firewall logs, and Zero Trust Access events, security analysts can significantly reduce Mean Time to Detect (MTTD) and effectively unmask sophisticated, multi-layered attacks.

Read on to learn more about how Log Explorer gives security teams the ultimate landscape for rapid, deep-dive forensics.

The flight recorder for your entire stack

The contemporary digital landscape requires deep, correlated telemetry to defend against adversaries using multiple attack vectors. Raw logs serve as the "flight recorder" for an application, capturing every single interaction, attack attempt, and performance bottleneck. And because Cloudflare sits at the edge, between your users and your servers, all of these events are logged before the requests even reach your infrastructure.

Cloudflare Log Explorer centralizes these logs into a unified interface for rapid investigation.

Zone-Scoped Logs

Focus: Website traffic, security events, and edge performance.

HTTP Requests

As the most comprehensive dataset, it serves as the "primary record" of all application-layer traffic, enabling the reconstruction of session activity, exploit attempts, and bot patterns.

Provides critical evidence of blocked or challenged threats, allowing analysts to identify the specific WAF rules, IP reputations, or custom filters that intercepted an attack.

DNS Logs

Identify cache poisoning attempts, domain hijacking, and infrastructure-level reconnaissance by tracking every query resolved at the authoritative edge.

NEL (Network Error Logging) Reports

Distinguish between a coordinated Layer 7 DDoS attack and legitimate network connectivity issues by tracking client-side browser errors.

For non-web applications, these logs provide visibility into L4 traffic (TCP/UDP), helping to identify anomalies or brute-force attacks against protocols like SSH, RDP, or custom gaming traffic.

Track and audit unauthorized changes to your site's client-side environment such as JavaScript, outbound connections.

Examine how third-party tools and trackers are interacting with user data, which is vital for auditing privacy compliance and detecting unauthorized script behaviors.

Account-Scoped Logs

Focus: Internal security, Zero Trust, administrative changes, and network activity.

Tracks identity-based authentication events to determine which users accessed specific internal applications and whether those attempts were authorized.

Provides a trail of configuration changes within the Cloudflare dashboard to identify unauthorized administrative actions or modifications.

CASB Findings

Identifies security misconfigurations and data risks within SaaS applications (like Google Drive or Microsoft 365) to prevent unauthorized data exposure.

Magic Transit / IPSec Logs

Helps network engineers perform network-level (L3) monitoring such as reviewing tunnel health and view BGP routing changes.

Tracks user actions inside an isolated browser session (e.g., copy-paste, print, or file uploads) to prevent data leaks on untrusted sites

Details the security health and compliance status of devices connecting to your network, helping to identify compromised or non-compliant endpoints.

DEX Application Tests

Monitors application performance from the user's perspective, which can help distinguish between a security-related outage and a standard performance degradation.

DEX Device State Events

Provides telemetry on the physical state of user devices, useful for correlating hardware or OS-level anomalies with potential security incidents.

```
}
{
```

Leveraging Commerce Data for Outcome-Based Relevancy in Agentic Recommendation Systems

Criteo Tech · Criteo Engineering

Author: Maxime Vono

Contributions from Thibault Becker , Hamlet Jesse Medina Ruiz , and Otmane Sakhi .

TL;DR

Most agentic commerce recommendation services today rely on content embeddings or mainstream LLMs (e.g., Claude, Llama, GPT). While effective for semantic understanding, these models are not grounded in real shopping behavior and are not optimized for commercial objectives such as clicks or conversions.

In this article, we show that leveraging commerce data at both the retrieval and re-ranking stages leads to substantial outcome-based relevancy gains :

+60% uplift in outcome-based relevancy for SKU re-ranking when incorporating transaction-derived popularity signals.

+37% uplift in outcome-based relevancy for SKU retrieval using a content embedding fine-tuned on organic clicks, compared to state-of-the-art zero-shot text encoders (MiniLM-L12, Gemma, Qwen).

These results highlight the importance of commerce-trained models for building high-performing agentic recommendation systems.

Disclaimer: Nike and Hoka brands have been chosen for illustrative purpose only.

Agentic commerce recommendation services are tailored to recommend the best products fitting a user query.

Metrics of interest

Throughout this article, we use relevancy in a sense that goes beyond surface-level semantic matching. Specifically, we focus on outcome-based relevance , commonly referred to in the recommendation systems literature as performance-based recommendation . Rather than assessing relevance solely through textual or semantic similarity between a query and product content, performance-based recommendation measures relevance through observable user outcomes, such as clicks, purchases, or downstream engagement. This distinction is critical in commerce settings, as it directly shapes how recommendation systems are evaluated and optimized.

Accordingly, there are two main routes used to evaluate agentic commerce recommendation services:

Accuracy (semantic relevance) answers the question: “Does this product description semantically match the user query?” (e.g., price, color, material). This metric has been used, for instance, to evaluate OpenAI Shopping Research , where 64% accuracy was reported. High accuracy can be achieved using retailers’ catalog data alone.

Outcome-based relevance answers the question: “Does this recommendation satisfy the user’s underlying need, as evidenced by real user behavior?” While semantic similarity is a necessary prerequisite for a good user experience, it is not sufficient. Two products may be equally accurate from a content standpoint (brand, category, attributes) yet differ significantly in their ability to meet user expectations, drive engagement, or lead to a purchase. Outcome-based relevance therefore represents a stronger and more user-centric notion of relevance , as it captures user intent through real commerce outcomes rather than inferred intent from text alone. As a result, commerce data — clicks, sales, and other behavioral signals — are essential to accurately measure and optimize relevance in agentic recommendation systems. Without grounding models in these outcomes, recommendation quality remains limited to proxy signals that fail to reflect what truly satisfies users.

Architecture of agentic commerce recommendation services

The most common and efficient architecture is a two-step one, detailed below:

Product retrieval happens first and involves getting a list of candidate products to ensure accuracy and capture one part of accuracy. It’s done using content embeddings and KNN search, optimised on commerce data.

Product re-ranking then orders the candidates products for maximizing outcome-based relevancy. It’s done using both query recommendation accuracy and commerce-driven scores, such as how much sales a given product drove. It also comes with recommendation explanations, sometimes coined reasoning .

We will deep-dive into the benefits of leveraging recommendation systems, optimized for outcome-based relevancy, into an agentic commerce recommendation service, compared to only using the content information of the returned products (e.g., title, category, or description). More precisely, we will present two benchmarks focusing on the product retrieval and re-ranking stages to illustrate the benefits of leveraging commerce data on top of content to target outcome-based relevancy for the end user.

Experimental Design. We conducted our evaluation using a dataset of user search queries paired with the corresponding clicked products. To evaluate outcome-based relevancy for the end user, we measured the ability to identify the product actually clicked for a given query from alternative, negative products. We used the actual rank of the clicked product for each user query to assess outcome-based relevancy, from a set of K = 400 products. We then normalized this rank to end up with the so-called reciprocal rank metric. A higher value for this metric indicates that users

```
}
{
```

Comments Considered Harmful in the Age of LLMs

Yegor Bugayenko

Writing code documentation is a pain. Not writing it leads to even bigger pain—we can't comprehend the code. However, writing it and then forgetting to update it causes the ultimate pain: it lies and confuses us. How about we cure all three pains at once: prohibit all comments! How do we know what the intent of the code is if we don't have any comments? We ask an LLM to explain it to us. What if the LLM fails to explain and confesses its inability? Then, we automatically fail the build and blame the author of the code. Thus, we introduce a new quality gate: Code Interpretability Score . The build passes only if this score is high enough.

The best minds in software engineering have long dreamed of self-documenting code. In 1974, Brian Kernighan and Phillip James Plauger said that "the only reliable documentation of a computer program is the code itself." In 2004, Steven McConnell in Code Complete claimed that "the main contributor to code-level documentation isn't comments, but good programming style." In 2008, Robert Martin in Clean Code suggested that "if our programming languages were expressive enough, or if we had the talent to subtly wield those languages to express our intent, we would not need comments very much—perhaps not at all." They all wanted the same thing: code that explains itself. They just lacked the tools to enforce it.

Why do we write comments at all? A Java method of a hundred lines may take hours to understand. A tiny Javadoc block saves this time:

class="highlight"> /** * Recursively finds the shortest * path between two nodes in the graph. */ int [] shortest (int [][] g , int a , int b) { /\/ A hundred lines of code go /\/ here, which we have no desire /\/ to read and understand. }

Comments promise to help us but fail in two distinct ways.

First, they are unclear . David Parnas once said that "documentation that seems clear and adequate to its authors is often about as clear as mud to the programmer who must maintain the code six months or six years later." What the author considers obvious, the reader finds cryptic.

Second, they decay . Being static metadata, comments do not evolve automatically with the code. If the implementation of the shortest() function stops being recursive, we may forget to update the Javadoc block. Such negligence leads to hallucinating documentation that causes bugs, broken trust, and wasted debugging time. In 1999, Andrew Hunt and Dave Thomas in The Pragmatic Programmer warned that " untrustworthy comments are worse than no comments at all." A recent analysis of 13 open source projects demonstrated that out-of-date comments are not rare but common.

Now we have a tool that solves both problems: the LLM.

}

Instead of writing the Javadoc block manually, we let the IDE generate it on-demand. The LLM reads the hundred lines of code, comprehends it, and summarizes the intent in a single English sentence. Modern models accomplish this task better than most humans. The documentation is always fresh because it is generated from the current code, not from a stale comment written months ago.

But we can go further. We can integrate an LLM into the build pipeline and ask it to assess the Code Interpretability Score (CIS) of every function. If the model has low confidence in explaining the logic, this signals that the code is too clever or convoluted. The compiler can enforce a threshold: if the CIS is too low, the build fails. This transforms readability from a subjective preference into an objective, measurable quality gate.

Once this gate exists, manual comments become not just unnecessary but harmful . They introduce a second source of truth that can contradict the code. The logical conclusion: prohibit them entirely. This forces developers to write clean, structured logic that is inherently machine-interpretable.

Robert Martin wished for more expressive languages. He didn't know about LLMs. Today, we don't need better languages—we need an LLM that can interpret any language. If the LLM can't explain the code, we blame the programmer and stop the build.

We are thinking about making EO , our experimental object-oriented language, this restrictive.

{

Programmers, Don’t Use Windows!

Yegor Bugayenko

In 2020, in the Junior Objects book I wrote this: “ Windows is not suitable for programmers. If you meet anyone who will tell you otherwise, you must know that you deal with a bad programmer, or a poor one, which are the same things. Your computer has to be MacBook. ” Now, five years later, I still hold the same opinion. This blog post is supposed to be less opinionated and, because of this, more convincing. The point is still the same: you either use Windows or you are a professional programmer.

First things first. This is what ChatGPT thinks about macOS vs. Windows (I toned it down a bit and sorted by importance, keeping what matters most at the top):

It’s POSIX -compliant

Tools like grep , awk , sed , ssh , and make work natively

Proper compiler toolchain: Clang, LLVM, make, git

Install everything with HomeBrew , one command away

Node, Python, Ruby, Go, Java—just work without PATH hell

The iTerm2 doesn’t look like it was built in 1998

Docker runs faster and cleaner than on Windows

SSH keys integrate smoothly with the system keychain

Git behaves predictably; no CRLF vs LF nightmares

I can hear you saying: What do I need it to be POSIX-compliant, and what is POSIX? Why do I need grep , sed , and awk ? Am I a 60 years old Unix admin? Why would I ever need git and make in the command line? I don’t use command line at all. I stay in the VS Code that works like a charm and helps me make a living.

I hear you. I do.

Now, hear me out. You are not a programmer. You look like one. You walk like one. You click the same buttons programmers click. You even make the same salary they make. But you are not one of them. Yet. Now, read on.

id=“what-is-unix”>What Is Unix?

Programmers are the masters of computers. They tell machines what to do. To simplify the task of managing a complex hardware, programmers invented a few layers of abstractions. The first layer is an operating system. Instead of dealing with the hard drive and the pixels on the screen directly, programmers invented files and stdout.

They did it in the Bell Labs , during the late 1960s and early 1970s. Earlier operating systems, like CTSS and OS/360 , gave them a good start. Unix was the first OS to say that everything is a file , including devices, directories, sockets, and processes. They also invented pipelines and the philosophy: “Write programs that do one thing well, and work together.” They also invented processes and their forking mechanism.

Their names were Ken Thompson and Dennis Ritchie .

id=“what-is-windows”>What Is Windows?

Five years later, another operating system was created, with different abstractions. Not everything was a file anymore, processes were not parallel, and there were no pipelines. The name of the system was CP/M and the name of the inventor was Gary Kildall . Then, five years later, 24-year-old Tim Paterson has made a copy of CP/M and called it 86-DOS . Microsoft purchased a non-exclusive license, rebranded it MS-DOS , and sold it to IBM. That’s how Windows was born, in 1981.

Why were there no proper files, no processes, and no pipelines? Because they weren’t trying to build a “real” operating system. CP/M and MS-DOS were designed for tiny, single-user, single-task microcomputers, not multi-user minicomputers or mainframes. Unix came out of Bell Labs—researchers, not hobbyists. CP/M and MS-DOS were made for personal computers: offices and home users. In other words, MS-DOS never meant to be a proper OS . It was something that can boot up a small machine and run a single program.

Then, in 1985, Windows 1.0 was built. It was a fancy GUI on top of MS-DOS, not a new OS. Later, in 1995, Microsoft introduced 32-bit APIs (Win32) and preemptive multitasking. However, the DOS subsystem was still lurking underneath. Windows 95 looked modern but was still a half-DOS zombie.

At the same time, in 1993, the team of Dave Cutler has built Windows NT that was not based on DOS at all. Latest Windows versions are descendants of NT, not MS-DOS. Under the hood it’s conceptually closer to Unix than to CP/M. There are features like protected memory, kernel/user separation, and file handles. However, still it’s not Unix.

id=“what-is-macos”>What Is macOS?

In 1984, Apple shipped their first Macintosh with the “System 1” operating system. It was no better than MS-DOS: no multitasking, no memory protection, and primitive file system. No surprise, it didn’t fly.

In 1997, Apple bought NeXT and adopted NeXTSTEP operating system. They made it the foundation for the new Mac OS —codenamed Rhapsody , later “Mac OS X”.

In 2001 they shipped Mac OS X 10.0 (“ Cheetah ”). Five years later I threw away my ThinkPad with Windows and bought my first MacBook with Mac OS X Leopard .

Modern macOS (Catalina, Ventura, Sequoia, etc.) is still built on that NeXT foundation. It is POSIX-compliant and, of course, it has processes and pipelines. In other words, it

}

standard-article(title: [Search Box and Cloud Function], author: [Ariya Hidayat], source-name: [Ariya Hidayat], images: (), paragraphs: ([For a blog hosted with Firebase Hosting, it turns out that a little search box is fairly easy to implement by using Cloud Functions for Firebase.], [As with the current trend nowadays, this blog is a static site prepared with Hugo and deployed to Firebase (see my previous blog: Static Site with Hugo and Firebase). Some time ago, I realized that since I am using Firebase anyway, might as well take advantage of its Cloud Functions to add a little search functionality to the blog, particularly for its 404 page .], [Of course, I am cheating a little bit. Using the above search box actually just redirects the search to my favorite search engine, DuckDuckGo , resulting in the following:], [Implementing it is almost trivial. First, we need index.js inside the functions subdirectory with the content as short as this (obviously, for your blog, replace site accordingly):], [const functions = require('firebase-functions'); exports.search = functions.https.onRequest((request, response) => { const q = request.query.q || ''; response.redirect(`https://duckduckgo.com/?q=site:ariya.io+${q}`); });], [Once it is properly deployed, the trigger URL will be in the form of us-central1-YOURFIREBASEPROJECT.cloudfunctions.net/search . This is rather ugly. To overcome that, set up a rewrite inside firebase.json so that it looks something like:], [{ "hosting": { "rewrites": [{ "source" : "/search", "function": "search" }] } }], [and thus, the function is available as the top-level /search of your Firebase Hosting URL, including if it is a custom domain.], [After this, inserting the search box is also equally fun:], [When a visitor uses the search, they will get redirected to DuckDuckGo and be presented with the search result. Fast and easy!],), insert-map: (:), word-count: 270, edited-for-length: false, debug-mode: false,)

standard-article(title: [I got tired], author: [Scott Hanselman], source-name: [Scott Hanselman], images: (), paragraphs: ([I have been blogging here for the last 20 years. Every Tuesday and Thursday, quite consistently, for two decades. But last year, without planning it, I got tired and stopped. Not sure why. It didn't correspond with any life events. Nothing interesting or notable happened. I just stopped.], [I did find joy on TikTok and amassed a small group of like-minded followers there. I enjoy my YouTube as well, and my weekly podcast is going strong with nearly 900 (!) episodes of interviews with cool people. I've also recently started posting on Mastodon (a fediverse (federated universe)) Twitter alternative that uses the ActivityPub web standard . I see that Mark Downie has been looking at ActivityPub as well for DasBlog (the blog engine that powers this blog) so I need to spend sometime with Mark soon.], [Being consistent is a hard thing, and I think I did a good job. I gave many talks over many years about Personal Productivity but I always mentioned doing what "feeds your spirit." For a minute here the blog took a backseat, and that's OK. I filled that (spare) time with family time, personal projects, writing more code, 3d printing, games, taekwondo, and a ton of other things.], [Going forward I will continue to write and share across a number of platforms, but it will continue to start here as it's super important to Own Your Words . Keep taking snapshots and backups of your keystrokes as you never know when your chosen platform might change or go away entirely.], [I'm still here. I hope you are too! I will see you soon.], [Related Links:], [Do they deserve the gift of your keystrokes?], [Do you have a digital or social media will?], [], [style="clear: both; padding-top: 0.2em;">],), insert-map: (:), word-count: 299, edited-for-length: false, debug-mode: false,)

onRequest((request, response) => { const q = request.

Ariya Hidayat

{

Not Everything is an Agent

Ariya Hidayat

“Agent” is likely going to be the word that will cause existential dread to true LLM enthusiasts.

Everyone’s got a different idea of what it means. In our modern age of innovation theater, lots of organizations gleefully slap the “agentic” label on anything that vaguely resembles a regular program (and pocket tons of money). Even a simple HTTP call to an LLM-as-a-Service can be called an agent, if you try desperately hard enough.

The internet, as always, is flooded with “groundbreaking” tutorials on building these so-called agents. Often authored by the latest hypefluencers , they typically involve a few lines (probably generated by whatever coding assistant is currently trending on Hacker News) that compose LangChain and an Ollama instance, often being presented as the pinnacle of AI autonomy. Because why bother with actual innovation when you can just repeat the quasi-boilerplate code ad nauseam ?

That’s why I liked it a lot when the Anthropic article, Building effective agents , came out, as it dares to suggest that simply bolting on retrieval or memory to an LLM does not , in fact, make an agent. And chaining or routing? That’s just glorified control flow , folks. Only when an LLM is tasked with truly complex, real-world tasks such as coding or using a computer, does it begin to resemble the autonomous agent we’ve been promised

So how do you identify a real agent? Don’t be fooled by the grand pronouncements of those rearranging deck chairs on the Titanic. Ask for the receipts of successful evaluations! Anecdotal evidence of a few successful LLM calls isn’t that useful. Remember, in the world of LLMs, as in life, the loudest claims are often the emptiest!

The internet, as always, is flooded with “groundbreaking” tutorials on building these so-called agents.

Ariya Hidayat

}

{

BRIEFS

IN BRIEF

Patrick McKenzie (patio11), patio11 (Patrick McKenzie): A history of an anomaly in the 2020 Coronavirus Pandemic, an independent research project about it, and what we need to learn about our next steps.

Patrick McKenzie (patio11), patio11 (Patrick McKenzie): Apple’s App Store is reforming their guidelines about taking out-of-band payments. This will be very interesting for developers, especially in the games industry.

Patrick McKenzie (patio11), patio11 (Patrick McKenzie): VaccinateCA, the non-profit I have been running, expanded nationally to Vaccinate The States . Here’s what we’ve learned in the last 100 days.

ZDNet, ZDNet: Amazon’s Big Spring Sale has plenty of cheap deals on useful tech gadgets this weekend. Save up to $25 on devices from Blink, Logitech, and more right now.

}

The Mirror

Vol. 1, No. 037 · 2026-03-30

Curated and composed automatically.